Bonsai Benchmark Methodology and Bottlenecks Found

Initial benchmark report - Oriol Pont, June 2026

Starting point

As described in the root README, the project began with Bonsai-family 1-bit language models as the acceleration target. Bonsai-1.7B is attractive because it is a general-purpose compact LLM, not just a toy benchmark or architectural demo, and its Q1_0 weights expose a hardware-relevant computation pattern: packed one-bit signs, per-group scales, and many repeated fixed-weight linear layers.

After several early experiments, the benchmark is organized in *tiers* according to the level of hardware abstraction and source code used. The first tier uses upstream `llama.cpp` as the known-correct software reference, because it provides the full Bonsai GGUF loading path, tokenizer/runtime support, and a practical CPU baseline for edge-style deployment. This tier answers where Bonsai spends time during real inference by instrumenting the `llama.cpp`/GGML CPU path and grouping profiled operator time into Q1_0 matrix operations, attention/KV-cache work, and other runtime work across increasing prompt lengths.

The second tier uses a self-contained explicit C++ runner for the same Bonsai GGUF model. Unlike `llama.cpp`, this runner is written as readable inference code with clear backend functions. It is used to extract full-model call counts and operation shapes that can later be mapped onto hardware experiments.

The third tier, developed after the software profiling, uses early software implementations of the two accelerator targets to measure their unaccelerated execution cost on a simulated NEORV32 CPU, to later serve as a baseline for hardware acceleration comparisons. Together, the tiers separate three questions: where the real model spends time when run on CPU (to identify bottlenecks), how often each backend structure appears in the full model, and how many NEORV32 software cycles each target operation costs before hardware acceleration.

Tier 1: llama.cpp CPU profiling

The first tier is the full software baseline, running Bonsai-1.7B Q1_0 GGUF through the upstream `llama.cpp` execution path, and profiling where time is spent inside the CPU operator graph. This is the reference used to decide which parts of inference are worth turning into hardware targets.

Benchmark setup

- Model: `Bonsai-1.7B-Q1_0.gguf`.
- Runtime: patched `llama.cpp`, using the `llama-batched-bench` benchmark executable, built from `batched-bench.cpp`.
- Machine: MacBookPro with Apple M1 Pro.
- Backend: CPU-only `llama.cpp` build with Metal/GPU disabled, Accelerate/BLAS enabled.
- Threading: 6 benchmark threads for prompt and decode runs.

The result files for these tables are, using “pp” prompts processed, and “tg” tokens generated:

- `results/tier1_llama_cpp_benchmark/full/prefill-summary.csv`
- `results/tier1_llama_cpp_benchmark/full/decode-summary.csv`
- `results/tier1_llama_cpp_benchmark/full/pl_1_pp_32768_tg_32.log`, as an example output for a specific run

The percentages in this tier are shares of profiled operator time. Prefill rows measure prompt processing, while decode rows measure autoregressive token generation as the KV cache grows. The profiling patch accumulates elapsed time inside GGML CPU/BLAS operators, and the summary scripts group that time into three buckets:

- `Q1_0`: time in `MUL_MAT` operations where the source weight tensor is Q1_0.
- `Attention`: time in `FLASH_ATTN_EXT`, used as the profiling proxy for attention and KV-cache traffic.
- `Other`: remaining profiled operator time.

The full sweep can be reproduced with `src/tier1_llama_cpp_benchmark/run-full-benchmark.py` after the `llama.cpp` setup step. The commands and setup details are documented in `src/tier1_llama_cpp_benchmark/README.md`.

Benchmark results

Prefill summary

Prompt	Tok/s	Q1_0 %	Attn. %	Other %
128	244.45	**88.24**	4.90	6.87
512	407.93	**81.75**	12.70	5.55
2048	343.69	**63.96**	31.74	4.30
4096	269.22	**49.86**	46.93	3.21
8192	155.64	31.33	**63.11**	5.56
16384	108.62	20.12	**78.16**	1.72
32768	56.10	10.70	**88.01**	1.29

Short and medium prompt prefill is dominated by Q1_0 matrix work. At 128 and 512 prompt tokens, Q1_0 operations account for more than 80% of profiled operator time. As the prompt grows, attention/KV-cache work rises steadily, and by 8192 tokens and above attention becomes the dominant prefill cost.

Decode summary

Prompt	Tok/s	Q1_0 %	Attn. %	Other %
0	63.28	**93.03**	2.80	4.17
128	63.20	**90.66**	6.49	2.85
512	48.05	**75.61**	15.51	8.87
2048	35.14	**54.94**	42.44	2.61
4096	14.16	37.50	**50.48**	12.02
8192	11.27	34.79	**64.87**	0.34
16384	5.43	12.30	**82.87**	4.83
32768	0.02	3.40	**94.74**	1.86

Early decode is also Q1_0-heavy, because each new token applies the fixed model weights while the KV history is still short. Long-context decode shifts toward attention/KV-cache traversal.

Tier 1 conclusions: a dual-target approach

The measured split identifies two clear, different regimes. At short contexts, where the model repeatedly applies fixed Q1_0 weights and attention has little history to read, the dependence is primarily arithmetic and packed weight processing: extracting one-bit signs, applying group scales, accumulating dot products, and writing output activations across many layers. On the other hand, at long contexts, attention becomes dominant. This dependence is mainly memory traffic and data movement: the KV cache grows with sequence length, and each generated token must access a larger history.

Since we aim to increase throughput at all different context sizes, this first benchmark already suggests embracing a dual-target approach. Before assessing the SoC restrictions and architecture, the project should treat these as separate bottlenecks with different causes: one mostly arithmetic over packed weights, the other mostly memory bandwidth and cache traversal.

Tier 2: self-contained Bonsai C++ runner

Tier 1 is the best reference for real CPU runtime behavior and understanding which were the main bottlenecks. For hardware co-design, a more direct source view is useful because the relevant work in `llama.cpp` is spread through the runtime, GGML graph execution, tensor abstractions, backend dispatch, tokenizer/runtime setup, and optimized kernels, which is complex and time-consuming. The project therefore also needs a simpler program, more adequate to the scope of the course project, that still runs the real Bonsai tensors and exposes the inference path as explicit loops and named backend functions.

The Tier 2 runner is kept under `src/tier2_explicit_runner/`. The main source file is `src/tier2_explicit_runner/bonsai-explicit-runner.cpp`, and the build/run commands are documented in `src/tier2_explicit_runner/README.md`. This tier is used to inspect the GGUF tensors and extract explicit operation metrics from a self-contained C++ Bonsai forward (decode) pass, including Q1_0 matrix-vector call counts, rows, dot-product elements, groups of 128 packed weights, attention calls,…

Benchmark setup

- Benchmark script: `src/tier2_explicit_runner/run-benchmark.py`.
- Input: explicit token ids, passed with `--tokens`, so this tier does not depend on tokenizer behavior.

The result files for this tier are:

- `results/tier2_explicit_runner/full/decode-summary.csv`
- `results/tier2_explicit_runner/full/check-q1.log`
- `results/tier2_explicit_runner/full/tokens_1.log`
- `results/tier2_explicit_runner/full/tokens_2.log`
- `results/tier2_explicit_runner/full/tokens_4.log`

The full Tier 2 run can be reproduced with:

```
python3 src/tier2_explicit_runner/run-benchmark.py
```

Decode metric results

Tokens	Layers	Q1 calls	Q1 dot elems	Q1 groups	Attn. calls	Attn. MACs
1	28	197	1.720B	13.437M	28	0.115M
2	56	394	3.440B	26.874M	56	0.344M
4	112	788	6.880B	53.747M	112	1.147M

For each decoded token, the full Bonsai path performs:

- 28 transformer layers.
- 196 transformer Q1_0 matrix-vector backend calls, plus 1 Q1_0 LM-head call.
- 1,719,904,256 total Q1_0 dot-product elements.
- 13,436,752 groups of 128 packed Q1_0 weights.
- 28 attention backend calls.
- 141 RMSNorm operations, 56 residual adds, and 28 SiLU-gate products.

The Q1_0 work is almost perfectly constant per decoded token because every new token runs through the same fixed Bonsai weights. The attention count also scales with layers and tokens, but the amount of attention work per token increases with the current KV-cache length. In the 1-token run, total attention score/value work is 114,688 MACs. In the 4-token run, total attention score/value work is 1,146,880 MACs, because the later tokens attend over a longer stored history.

Tier 2 conclusions

The main result from this tier is a bridge from the full `llama.cpp` timing benchmark to hardware-sized kernels: the Q1_0 accelerator should be evaluated on packed 128-weight groups and row-wise dot products, while the attention accelerator should be evaluated on KV-cache traversal and per-head score/value reductions as context grows. The authored self-contained runner also provides a simpler, clearer source view of the Bonsai inference path, from which to design the hardware accelerator and its interface to the memory hierarchy.

Tier 3: NEORV32 software baselines

Tier 3 moves the two target operations onto the intended processor architecture while keeping them entirely in software. These are pre-acceleration reference measurements: the kernels run on the NEORV32 CPU in GHDL simulation and use the CPU cycle counter around the operation only. Input preparation and UART result reporting are outside the measured region, to avoid including unrelated software overhead.

For both benchmarks (one for each of the two accelerator targets), the project provides two profiles, which serve different purposes:

- `board`: uses the project's board-target NEORV32 configuration, with all measured code and data placed in 16 KiB of CPU-local instruction memory and 8 KiB of CPU-local data memory. The Tang Nano 9K also provides 8 MiB of PSRAM, reserved for later implementation and evaluation of the hardware memory path.
- `bonsai`: uses enlarged simulated memories to exercise a more representative Bonsai operation shape or exported model fixture. It still may not be an exact Bonsai model, given that it would cause the simulation to run out of memory or run too slowly.

The simulation logs and build-size reports are stored beside each summary. The reproducible commands, runner alternatives, and profile definitions are documented in `src/tier3_neorv32_cycle_kernels/README.md`.

Q1_0 by Q8_0 matrix-vector baseline

This kernel follows the GGML-style integer dot-product path used at the Tier 1 boundary and made explicit in Tier 2: activations are supplied as Q8_0 blocks, fixed weights as packed Q1_0 groups, and each output row accumulates Q1_0 by Q8_0 integer dot products with their scales. The board profile measures one native 128-weight group, while the Bonsai profile measures one 2048-element hidden-width row using packed weights and expected values exported from the GGUF fixture.

The result files are in:

- `results/tier3_neorv32_cycle_kernels/q1_matvec/board/summary.csv`
- `results/tier3_neorv32_cycle_kernels/q1_matvec/bonsai/summary.csv`

Profile	Rows	Columns	Input	Cycles	Cycles/group	Cycles/elem.
Board	1	128	Synthetic	7,934	7,934.00	61.98
Bonsai	1	2,048	GGUF fixture	195,602	12,225.13	95.51

The board result is the smallest complete accelerator work unit and establishes the cost of processing one packed Q1_0 group in CPU software simulation. The Bonsai result contains 16 such groups and establishes the cost of a representative row. Future accelerator measurements will report the same operation counts and compare kernel-only cycles against these baseline values.

Attention/KV software-service baseline

The second kernel implements the attention service expected at the accelerator boundary: append the current K/V vectors, map query heads to grouped-query KV heads, scan K to compute scaled QK scores, apply stable exact softmax, scan V for the weighted accumulation, and produce the attention output. Its phase and service counters measure complete CPU software execution, including the ordinary loads and stores used to access K/V data. They therefore establish the combined pre-acceleration software cost of each phase. Later hardware simulations will separate engine compute-active cycles from memory/FIFO wait cycles, allowing compute acceleration and memory-path improvements to be evaluated independently.

The result files are:

- `results/tier3_neorv32_cycle_kernels/attention_kv/board/summary.csv`
- `results/tier3_neorv32_cycle_kernels/attention_kv/bonsai/summary.csv`

Profile	Q heads	KV heads	Head dim	Ctx	Score MACs	Value MACs	KV bytes	Service cycles
Board	1	1	32	2	64	64	384	489,007
Bonsai GQA	2	1	16	2	64	64	320	494,741

Profile	Append cycles	Score cycles	Softmax cycles	Value cycles
Board	466	259,391	5,493	223,419
Bonsai GQA	183	257,965	11,067	224,674

Both profiles deliberately perform 64 score MACs and 64 value MACs, and their service totals are correspondingly close. The score and value reductions dominate execution, with exact softmax accounting for only a small share at a 2-token context length. This agreement is useful as a validation of the operation and counter contract, yet `ctx = 2` is not a long-context bandwidth benchmark, which is too expensive to run in simulation (at least in this early benchmarking stage).

The byte counters describe logical K/V traffic implied by the operation. True separation of compute-active, FIFO-wait, buffer-wait, and total command cycles becomes possible only after the hardware interface and engine exist. The future comparison will thus first measure a straightforward hardware implementation and then retain its compute service while improving the stream/FIFO memory path.

Conclusions

The three tiers now form a direct measurement chain:

- Tier 1 identifies when Q1_0 computation or attention/KV work dominates real `llama.cpp` inference, identifying two separate bottlenecks as acceleration targets: Q1_0 matrix-vector work and attention/KV-cache traversal.
- Wrapping these two targets as external functions, Tier 2 supplies full-model call counts and operation dimensions, which will be useful for extrapolation from a single execution to how it would affect end-to-end inference performance.
- Tier 3 supplies simulated NEORV32 software cycle baselines for the services selected for acceleration, to serve as a baseline for later hardware acceleration comparisons.

This benchmark report is the first step in a co-design process, which directly informs the decisions done about the high-level architecture and interface of the Bonsai accelerator.